

Context-Gated Multi-Channel Hebbian Plasticity Collapses Under End-to-End Training

A negative-result technical report

Project repository: [Ynkling/clankers](#) • Draft

Abstract

We investigate an architectural idea: augment the biologically-inspired *Dragon Hatchling* (BDH) language-model architecture so that each synaptic connection carries *multiple* weight channels, with a learned, context-dependent gate selecting or blending channels per token. The motivation is that one network might then express several distinct “modes” of working memory—analogous to a brain synapse using multiple neurotransmitters—without separate mixture-of-experts modules. We derive the mechanism, implement it exactly (verifying our recurrent implementation against the reference architecture to machine precision), and then test whether the channels do anything useful. They do not. Across four experiments of escalating directness—a cross-context recall task, an interference task where channel separation is provably beneficial, an auxiliary-loss sweep explicitly rewarding separation, and a hard discrete-routing variant with a stability probe—the gate consistently collapses to permissive, near-uniform routing and never separates the channels. Critically, when the gate is *initialized* to the perfect separated solution, training *erodes* it: the separated configuration is not even a stable fixed point of the objective. We give a mechanistic explanation: the Hebbian write rule makes permissive routing locally optimal, so channel separation is anti-optimal under gradient descent. This is a small-scale but cleanly demonstrated negative result, connecting the known phenomenon of router collapse in conditional computation to a specific and previously undocumented cause—the interaction between routing and inference-time synaptic plasticity.

1 Background

1.1 The Dragon Hatchling (BDH) architecture

BDH [1] is a recently proposed language-model architecture designed as a bridge between the Transformer and biological models of the brain. Its distinguishing feature is that the model’s *working memory during inference* is stored in synaptic state that is updated by a Hebbian learning rule—“neurons that fire together wire together”—rather than in a fixed-length activation vector as in a recurrent network, or a growing key–value cache as in a Transformer.

The paper defines two related forms. The full **BDH** stores state as an $n \times n$ synaptic matrix σ , where n is a (large) number of neurons. The GPU-friendly **BDH-GPU**, which is the version implemented in the public code release, stores a mathematically equivalent but compressed state $\rho \in \mathbb{R}^{n \times d}$ with $d \ll n$, related to the full synaptic state by $\rho = E\sigma$ for an encoder matrix E . The core recurrence, following equation (8) of the BDH paper [1], is

$$\rho_{t,l} = (\rho_{t-1,l} + \text{LN}(E y_{t,l-1}) x_{t,l}^\top) U, \quad (1)$$

$$x_{t,l} = x_{t-1,l} + (D_x \text{LN}(E y_{t,l-1}))^+, \quad (2)$$

$$y_{t,l} = (D_y \text{LN}(\rho_{t-1,l} x_{t,l}))^+ \odot x_{t,l}, \quad (3)$$

where $x, y \in \mathbb{R}^n$ are sparse, positive neuron activations, $v^* \in \mathbb{R}^d$ is a compressed value vector, $(\cdot)^+$ is a ReLU, $\odot$ is elementwise product, LN is a parameter-free LayerNorm, and U is a diagonal or

block-diagonal matrix implementing positional decay/rotation (ALiBi or RoPE). The trainable parameters are the three matrices E, D_x, D_y .

The essential fact for this report is that the linear-attention operation in the public code is *identical* to a token-by-token Hebbian accumulation of the state matrix. The state at time t can be written explicitly (equation (16) of [1]) as a sum of outer products over the past:

$$\rho_{t-1,l} = \sum_{\tau < t} v_{\tau,l-1}^* x_{\tau,l}^\top U^{t-\tau}, \quad a_{t,l}^* = \rho_{t-1,l} x_{t,l}. \quad (4)$$

Reading memory is $a_t^* = x_t^\top$ applied to the accumulated state; writing memory is adding the outer product $v_t^* \otimes x_t$ at each step. This outer-product write *is* the Hebbian rule.

1.2 The proposed extension: multiple channels per synapse

The idea studied here is to give the synaptic state k independent channels. Concretely we maintain a state tensor $S \in \mathbb{R}^{k \times n \times d}$, a learned gate projection $W_g \in \mathbb{R}^{d \times k}$, and for each token a gate distribution

$$g_t = \text{softmax}(v_t^* W_g) \in \Delta^{k-1}, \quad (5)$$

a probability vector over the k channels. The Hebbian write deposits each token’s outer product into the channels in proportion to the gate,

$$S += \sum_c g_t[c] (x_t \otimes v_t^*) \quad (\text{one slice per channel } c), \quad (6)$$

and the read blends the per-channel outputs by the same gate,

$$a_t^* = \sum_c g_t[c] (x_t^\top S_c). \quad (7)$$

Working through the algebra, the effective attention score between a query token t and a past key token τ becomes

$$\underbrace{(x_t \cdot x_\tau)}_{\text{content match}} \times \underbrace{(g_t \cdot g_\tau)}_{\text{context-gate match}}. \quad (8)$$

The content term is ordinary BDH attention; the new gate term modulates it. The design intent is that tokens sharing a context (similar g) can read each other’s memory, while tokens in different contexts (near-orthogonal g) are isolated into separate channels. When $k = 1$ the gate is trivially the scalar 1 and the mechanism reduces exactly to single-channel BDH.

1.3 Relationship to prior work

The blending mechanism itself is not novel. Computing a weighted combination of a bank of weight matrices with input-dependent coefficients is the core of CondConv [2], and closely related ideas appear as parameter superposition [3] and context-dependent gating [4]. All of these fall under the umbrella of hypernetworks / conditional weight generation. What is specific to the present work—and what makes the negative result below non-obvious—is the combination of context-dependent routing with *inference-time Hebbian plasticity*: unlike CondConv, whose weight banks are frozen, here each channel carries a state that *evolves during the forward pass*. It is precisely this interaction that produces the failure we document. Finally, the collapse we observe is an instance of the well-known *router collapse* phenomenon in mixture-of-experts models [5]; our contribution is to identify a novel driver of it.

2 Implementation and verification

Before testing whether the mechanism helps, we verified that our implementation computes what the equations specify. This matters because the recurrent (token-by-token) form used for the experiments must match the parallel (token-parallel) form that the reference code and the paper’s Appendix E define.

We implemented both forms and checked their outputs agree. All checks use double precision; the reference threshold for “machine precision” is $\sim 10^{-15}$.

Verification check	Max abs. difference
Single-channel recurrent vs. parallel reference (Appendix E)	1.78×10^{-15}
Multi-channel ($k=3$) parallel vs. recurrent	8.88×10^{-16}
$k=1$ reduction to single-channel (parallel path)	0 (bitwise identical)
$k=1$ reduction to single-channel (recurrent path)	1.78×10^{-15}

Table 1: The recurrent multi-channel implementation matches the reference architecture to machine precision, and reduces exactly to single-channel BDH at $k = 1$. The bitwise-identical parallel reduction follows because at $k = 1$ the softmax returns exactly 1.0 and the gate outer product is an all-ones matrix, leaving the computation path algebraically unchanged.

Two limitations of the verification should be stated plainly. First, the equivalence was checked with $U = I$ (no positional decay or rotation); this is a valid instance of the paper’s equations and isolates the Hebbian identity cleanly, but the folding of RoPE/ALiBi into the recurrence is structurally standard rather than bit-verified here. Second, the full $n \times n$ synaptic matrix σ is only materialized at small n for analysis; at scale one trains on the compressed ρ and recovers σ slices via equation (4) only for interpretability. Neither limitation affects the conclusions below, which concern the compressed, trainable form.

3 Experiments

All experiments use a small model ($d = 32$, $n = 64$, byte-level vocabulary) so that each run completes in minutes on CPU, and every result is reported as mean $\pm$ standard deviation over 3 random seeds. Each experiment isolates the loss to the specific token positions where the effect under test would appear (the query positions), rather than averaging over whole sequences.

3.1 Experiment 1: cross-context recall

We first asked whether the channels separate when nothing rewards separation. The task presents key–value associations under one context, then a query under a *different* context that can only be answered by recalling the earlier association. If the channels isolated the two contexts, the query could not read the earlier memory—so a working model here *must* keep memory shared.

Result. All conditions—single-channel, multi-channel with a learned gate, and multi-channel with a forced-uniform gate—achieved essentially perfect query accuracy (loss ≈ 0.0002 , accuracy = 1.000), indistinguishable at every checkpoint. Inspecting the learned gate showed it did *not* route the two contexts to different channels: the gate collapsed to near-uniform. The apparent conclusion “isolation is free” is misleading, because the isolation mechanism never engaged. The correct reading is that end-to-end training drives the gate toward permissiveness, since separation would only hurt on this task. This motivated a task where separation should *help*.

3.2 Experiment 2: interference (separation is provably beneficial)

We constructed a task where a single shared memory is provably insufficient. Two interleaved streams use the *same* key tokens but assign them *conflicting* values (stream 1: $X \rightarrow 5$; stream 2: $X \rightarrow 9$, for the same key X). A per-stream context signal drives the gate. At query time, “ $X=?$ ” must return 5 under the stream-1 context and 9 under the stream-2 context. With a single Hebbian memory, the two associations are written to the same synapses and average out—the model cannot exceed a hard ceiling. With separated channels, each stream’s association lives in its own state and both are recoverable. A sanity check confirmed the analytical single-channel maximum is 50% accuracy.

Condition	Stream-1 acc	Stream-2 acc	Lift fraction
Single-channel baseline	0.508 ± 0.032	0.493 ± 0.022	— (floor)
MC $k=2$, learned gate	0.502 ± 0.027	0.490 ± 0.026	−0.01
MC $k=2$, <i>perfect</i> gate (hand-set)	1.000 ± 0.000	1.000 ± 0.000	1.00 (ceiling)
MC $k=2$, uniform gate	0.508 ± 0.032	0.493 ± 0.022	0.00 (floor)

Table 2: Interference task. The “lift fraction” is $(\text{acc} - \text{floor})/(\text{ceiling} - \text{floor})$: 1.0 means the condition matches perfect separation, 0.0 means it matches no separation. A hand-set gate solves the task perfectly (ceiling = 100%), but the learned gate captures −1% of that benefit and sits exactly on the floor.

Result. The hand-set “perfect gate” condition confirms the architecture is fully *capable* of the task: with the right routing, accuracy is 100% where a single channel caps at 50%. Yet the *learned* gate captured none of this. The cosine similarity between the two streams’ gate vectors was 0.9993—the gate did not separate the streams at all, even though separation was the only path above 50%. The failure is not in the model’s representational capacity; it is in the learning dynamics.

3.3 Experiment 3: explicit auxiliary separation loss

We next added a loss term that directly rewards the gate for routing different-context tokens to different channels, scaled by how different their contexts are:

$$\mathcal{L}_{\text{aux}} = -\lambda \cdot \text{mean}_{i,j} \left[\|c_i - c_j\| (1 - \cos(g_i, g_j)) \right], \quad (9)$$

where c is the per-token context signal. We swept $\lambda \in \{0, 0.01, 0.1, 1.0\}$ —a hundredfold range.

Result. No value of λ produced separation or lift. The gate cosine stayed pinned near 1.0 across the entire sweep (ending at 0.9999 at $\lambda = 1.0$, marginally *more* collapsed than at $\lambda = 0$), and the lift fraction never left the floor (0.00 at best). Notably, the task loss did *not* degrade as λ increased—there was no over-forcing, meaning the auxiliary loss and the task loss were not in tension. The gate simply ignored a direct, strong incentive to separate. This indicates the collapse is not a shallow optimization artifact one hyperparameter away from a fix.

3.4 Experiment 4: hard discrete routing and a stability probe

Finally we replaced the soft gate blend with hard, discrete routing via a straight-through estimator: the forward pass commits to a single channel per token (a one-hot choice), while gradients flow through the underlying soft gate. Both the read and the Hebbian write use the hard choice. We ran two conditions: (1) standard random initialization, testing whether hard routing spontaneously separates; and (2) a gate *initialized* to the perfect separated solution, testing whether that solution is *stable* under training.

Condition	Stream-1 routing	Stream-2 routing	Lift
Discrete gate, standard init	ch0 0.41, ch1 0.59	ch0 0.52, ch1 0.48	−0.00
Discrete gate, <i>separated</i> init	ch0 0.64, ch1 0.36	ch0 0.30, ch1 0.70	−0.01

Table 3: Discrete-routing conditions. “Routing” is the fraction of each stream’s tokens sent to each channel; clean separation would read approximately (1.0, 0.0) for one stream and (0.0, 1.0) for the other. Standard init shows no stream/channel correspondence. Separated init—handed the perfect solution—drifts back toward mixing, and its lift returns to the floor.

Result. From standard initialization, discrete routing showed no reliable stream/channel correspondence and no lift—the same collapse as the soft case, reached by a different route. The decisive finding is condition 2: initialized to the perfect separated solution (the configuration that condition 3 of Experiment 2 proved achieves 100%), training *eroded* it, drifting from a clean split back toward mixing (0.64/0.36 and 0.30/0.70) with lift returning to −0.01. The separated solution is therefore not merely hard to *find*; it is not even a stable fixed point. Gradient descent actively moves away from it.

4 Why the mechanism collapses

The four experiments admit a single mechanistic explanation, rooted in the Hebbian write rule of equation (6).

Every token, whichever channel it is routed to, writes a genuinely useful outer-product memory into that channel. Consequently, spreading a stream’s tokens across *both* channels causes *both* channels to accumulate partial, usable memory of that stream. At read time, more of the total state is populated and available, so the loss is *lower* under permissive routing. Clean separation, by contrast, leaves half of the state cold for each stream—each channel only ever sees one stream’s writes—which is locally worse at read time even though it is what the interference task ultimately rewards.

The optimizer sees permissive routing as locally better *at every step*, from any starting point. This is why the gate collapses from random initialization (Experiments 1–3), why an explicit separation loss cannot dislodge it without also fighting the task loss it is not in tension with (Experiment 3), and—most tellingly—why a perfectly separated initialization is eroded rather than preserved (Experiment 4). The very property that makes the architecture biologically appealing, namely that every co-activation writes to the synapses, is exactly what makes channel separation anti-optimal under gradient descent. Routing and plasticity are in structural conflict: the router would need to withhold writes from all-but-one channel to create separation, but withholding writes is precisely what the plasticity-driven loss penalizes.

5 Conclusion

A context-gated multi-channel Hebbian architecture is expressible and provably capable: a hand-set gate solves a memory-interference task (100%) that a single channel caps at 50%. But end-to-end training never uses that capacity. The gate collapses to permissive routing across soft, auxiliary-loss-augmented, and hard-discrete variants, and the separated solution is not even a stable fixed point—gradient descent erodes it from a separated initialization. The cause is specific and, to our knowledge, previously undocumented in this form: the Hebbian write makes permissive routing locally optimal, so channel separation is anti-optimal under the training objective.

We report this as a negative result in the spirit of saving subsequent effort. The idea of multi-neurotransmitter-like synaptic channels is intuitive and recurs naturally; this report documents, with a clean ladder of experiments and a mechanistic account, why the straightforward end-to-end realization of it does not work on the tasks studied. We do not claim the result generalizes beyond the small scale, the single architecture family, and the task designs examined here—establishing its scope would require further work. But within that scope the finding is unambiguous and consistently reproduced.

A constructive reading points to where a future attempt would have to differ: any mechanism hoping to make plastic channels specialize under training must break the structural conflict identified in Section 5—for example by decoupling the write from the routing (so that unrouted channels are not penalized for staying empty), or by supplying a training signal that rewards separation strongly enough to overcome the plasticity gradient rather than merely competing with it. Whether such a mechanism exists, and whether it retains the interpretability that motivated the plastic substrate in the first place, is left open.

References

- [1] A. Kosowski, P. Uznański, J. Chorowski, Z. Stamirowska, M. Bartoszkiewicz. *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain*. arXiv:2509.26507, 2025.
- [2] B. Yang, G. Bender, Q. V. Le, J. Ngiam. *CondConv: Conditionally Parameterized Convolutions for Efficient Inference*. NeurIPS, 2019.
- [3] B. Cheung, A. Terekhov, Y. Chen, P. Agrawal, B. Olshausen. *Superposition of Many Models into One*. NeurIPS, 2019.
- [4] N. Y. Masse, G. D. Grant, D. J. Freedman. *Alleviating Catastrophic Forgetting using Context-Dependent Gating and Synaptic Stabilization*. PNAS, 2018.
- [5] N. Shazeer et al. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. ICLR, 2017.